Software Requirements Specification (SRS)

CRASH SENSE: AI-Based Application Crash Prediction & Prevention System

Document Control	**Group 8** **Muhammed Shafi A S** **Hasna Sherin C K** **Ananthu A R** **Madhav R**
Version	1.0
Status	**Initial Draft**
Date	January 14, 2026
Project Name	**CRASH SENSE**
Prepared For	Mini-Project

Table of Contents

Section	Title	Page Number
1	**Introduction**	**3**
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions and Acronyms	4
2	**Overall Description**	3
2.1	Product Perspective	3
2.2	User Classes and Characteristics	4
2.3	Operating Environment	4
3	**Functional Requirements**	5
3.1	Data Ingestion & Monitoring	5
3.2	Prediction & Detection	5
3.3	Recommendation & Remediation (New)	5
4	**Non-Functional Requirements**	**5**
4.1	Performance	5
4.2	Reliability	5
4.3	Usability	5
5	**External Interface Requirements**	**6**
5.1	Hardware Interfaces	6
5.2	Software Interfaces	6
6	**Appendix A: Glossary**	**6**
7	**Appendix B: Analysis Models**	**7**
8	**Appendix C: To Be Determined (TBD) List**	**8**

1. Introduction

1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for **CRASH SENSE**. This system is designed not only to detect application crashes but to **predict** them before they occur, identify the root cause, propose solutions, and—upon user approval—execute automated remedial actions to prevent system failure.

1.2 Scope

The **Crash Sense: AI-Based Application Crash Prediction & Prevention System** aims to overcome the limitations of existing crash detection tools by implementing a multi-modal, predictive, and explainable AI framework.

1.2.1 In-Scope Functions (Deliverables)

The system will encompass the following key functionalities:

- **Multi-Source Data Integration:** Unlike systems that rely solely on one data type, this project will simultaneously ingest and analyse two distinct data sources:
  - **System/Application Logs:** Text-based error logs and event streams.
  - **Runtime Performance Metrics:** Numerical data such as CPU usage, memory consumption, and I/O rates.
- **Advanced Data Synchronization:** The system will implement a preprocessing pipeline to align unstructured log data with structured performance metrics based on timestamps, ensuring a unified dataset for analysis.
- **Predictive Anomaly Detection:** The system will go beyond simple "post-failure" reporting. It will utilize Machine Learning models to identify pre-crash patterns and forecast potential failures *before* they result in a system shutdown.
- **Explainable Root Cause Analysis (XAI):** To address the "black box" issue of current models, the system will provide detailed explanations for every alert. This includes highlighting the specific log lines or metric spikes that contributed to the crash prediction.
- **Robust False Positive Reduction:** The model will be trained on diverse datasets to improve generalization and reduce false alarms, ensuring that alerts sent to developers are actionable and accurate.

1.2.2 Out-of-Scope (Exclusions)

To ensure the project remains feasible within the timeline, the following are excluded:

- **Automatic Remediation:** The system will detect and explain the crash, but it will *not* attempt to automatically patch the code or restart the server.
- **Hardware-Level Diagnostics:** The scope is limited to software-level logs and metrics; physical hardware faults (e.g., disk failure, overheating) are not monitored unless reflected in OS logs.

- **Real-Time Distributed Tracing:** The system focuses on single-node application monitoring rather than complex microservice distributed tracing across multiple servers.

1.3 Definitions and Acronyms

- **SRS:** Software Requirements Specification
- **Remediation:** The act of correcting a fault or deficiency.
- **RCA:** Root Cause Analysis.
- **HITL:** Human-in-the-Loop (User permission required for actions).
- **Target System:** The Linux environment (Physical Server, Cloud Instance, or Desktop) where the monitored application resides.

---

2. Overall Description

2.1 Product Perspective

CRASH SENSE acts as an active guardian rather than a passive monitor. It sits between the Operating System and the Target Application on standard **Linux environments**.

During the **development phase only**, the system is tested on **Virtual Devices** to safely simulate destructive crash scenarios (memory leaks, infinite loops, kernel panics) without risking physical hardware stability. The final product is intended for deployment on production Linux servers and workstations.

2.2 User Classes and Characteristics

- **Developer:** Uses the system to debug code and analyze crash reports.
- **System Administrator:** Deploys the agent on Linux servers, receives alerts, and authorizes automated fixes.

2.3 Operating Environment

- **Target Production Environment:**
  - **OS:** Linux (Ubuntu, CentOS, Debian, RHEL).
  - **Hardware:** Physical Servers, Cloud Instances (AWS EC2, Azure VM), or Edge Devices.
- **Development/Testing Environment:**
  - **Platform:** Virtual Devices / Virtual Machines (VirtualBox, VMware) for stress testing.
- **Software Stack:**
  - **Language:** Python 3.9+.
  - **Libraries:** scikit-learn, TensorFlow/Keras, Pandas, NumPy, SHAP.
  - **UI:** Flask/Streamlit.

---

3. Functional Requirements

3.1 Data Ingestion & Monitoring

- **FR-01:** The system shall collect CPU, Memory, and Disk I/O metrics every 500ms from the host Linux system.
- **FR-02:** The system shall parse application-specific logs in real-time.

3.2 Prediction & Detection

- **FR-03:** The system shall predict a crash at least 30 seconds before occurrence with 85% accuracy.
- **FR-04:** The system shall identify the "Leading Indicator" (e.g., Memory spike) using SHAP values.

3.3 Recommendation & Remediation (New)

- **FR-05:** The system shall maintain a database of "Crash Signatures" mapped to "Recommended Actions".
  - *Example:* Signature "OOM Error" -> Action "Clear Temp Cache" or "Restart App".
- **FR-06:** The system shall present the recommended action to the user via a prompt.
- **FR-07:** The system shall execute the remediation script (Bash/Python) **only** upon receiving boolean True (Permission Granted) from the user.
- **FR-08:** The system shall verify if the application stabilized after the action.

---

4. Non-Functional Requirements

4.1 Performance

- **NFR-01:** Prediction latency must be under 200ms to allow time for preventive action.
- **NFR-02:** The agent must be lightweight, consuming < 5% CPU, to run efficiently on both high-end servers and edge Linux devices.

4.2 Reliability

- **NFR-03:** The Automated Remediation module must have a "Safe Mode" that prevents executing destructive commands (e.g., rm -rf).

4.3 Usability

- **NFR-04:** Alerts must be color-coded (Yellow=Warning, Red=Critical) for immediate recognition.

---

5. External Interface Requirements

5.1 Hardware Interfaces

- **System Resources:** Direct interaction with Linux Kernel interfaces (e.g., /proc filesystem) to read hardware states (CPU temp, Fan speed) where available.

5.2 Software Interfaces

- **Notification System:** Email or Slack API for sending alerts.
- **OS Shell:** Bash/Zsh access for executing fix scripts.

---

6. Appendix A: Glossary

Term	Definition
Crash Signature	A specific pattern of metrics (e.g., high memory usage combined with low disk I/O) or log entries that historically precedes an application failure.
HITL	**Human-in-the-Loop**. A model requiring human interaction (specifically permission) before executing automated remediation scripts (FR-07).
Leading Indicator	A specific metric or feature (e.g., CPU load, thread count) identified by the model as the primary contributor to a predicted crash.
OOM	**Out of Memory**. A common error state where the system cannot allocate enough memory for the application, often leading to a crash.
RCA	**Root Cause Analysis**. The process of identifying the fundamental cause of a fault or failure (e.g., distinguishing between a memory leak and a database timeout).
Remediation	The act of correcting a fault. In this system, it refers to scripts (Bash/Python) executed to stabilize the application (e.g., clearing cache).
SHAP	**SHapley Additive exPlanations**. A game-theoretic approach to explain the output of any machine learning model, used here to explain *why* a crash is predicted.
Target System	The host Linux environment (Physical Server, Cloud Instance, or Edge Device) where the monitored application and the Crash Sense agent reside.
XAI	**Explainable AI**. Artificial intelligence in which the results of the solution can be understood by humans.

7. Appendix B: Analysis Models

The **Crash Sense** system utilizes a multi-modal analysis framework combining time-series forecasting with natural language processing.

1. Time-Series Anomaly Detection (Metric Analysis)

- **Purpose:** To analyze numerical data collected every 500ms (CPU, Memory, Disk I/O) as specified in FR-01.
- **Technique:** Long Short-Term Memory (LSTM) or similar Recurrent Neural Networks (RNN) are suitable for identifying sequential patterns and dependencies in time-series data.
- **Function:** Predicts future metric values and flags deviations that match known pre-crash patterns (FR-03).

2. Log Pattern Recognition (Text Analysis)

- **Purpose:** To parse and analyze unstructured application logs in real-time (FR-02).
- **Technique:** Natural Language Processing (NLP) techniques, potentially using tokenization and sequence classification to detect error keywords (e.g., "Exception", "Timeout", "Fatal") and their frequency.
- **Function:** Correlates log events with metric spikes to increase prediction confidence.

3. Explainability Model (SHAP)

- **Purpose:** To satisfy the "Glass Box" requirement (FR-04) by explaining prediction results.
- **Technique:** Kernel SHAP or Tree SHAP.
- **Function:** Assigns an importance value to each feature (e.g., "Memory Usage contributed 80% to this crash prediction"), allowing developers to trust and verify the alert.

---

8. Appendix C: To Be Determined (TBD) List

The following items are identified as necessary for the full production release but are currently undefined or pending detailed specification:

ID	Item	Description
TBD-01	Specific Remediation Scripts	The exact Bash/Python scripts for specific crash signatures (other than "Clear Cache" and "Restart App") need to be defined and safety-tested.
TBD-02	Alert Thresholds	The specific numerical thresholds for "Warning" (Yellow) vs. "Critical" (Red) alerts (NFR-04) need to be calibrated for different hardware types.
TBD-03	Data Retention Policy	How long logs and metric data should be stored on the local device before rotation or deletion has not yet been specified.
TBD-04	Supported Log Formats	The specific list of application log formats (e.g., JSON, Apache Common, Syslog) the parser supports out-of-the-box needs to be finalized.
TBD-05	API Security Protocols	Authentication mechanisms for the External Interface (Section 5.2) to prevent unauthorized remediation triggers.